Deliverable 1 — Requirements Document

Healthcare Web Application

Course Code — CSE 616
Course Name — Web Engineering Lab

Group Number: 14

Group Members:
Israt Zannat — 22701027
Sajid Shahriar Sanam — 22701031
Shajeda Isnat — 22701060

Course Instructor:
Professor Rokan Uddin Faruqui

Department of Computer Science and Engineering,
University of Chittagong

Date Prepared: September 17, 2025

Contents

1 Executive Summary **3**

2 Project Objectives **3**

3 Scope **3**

4 Actors and Roles **4**

5 Functional Requirements **4**

6 Non-Functional Requirements **6**

7 User Stories with Acceptance Criteria **7**

8 Technology Stack and Justification **10**

9 Team Roles and Responsibilities **11**

10 Project Timeline and Milestones **12**

11 Risk Assessment and Mitigation **13**

12 Setup Confirmation **14**

1 Executive Summary

This document presents Deliverable 1 for the Healthcare Web Application capstone project. It defines the project objectives, scope, actors, functional and non-functional requirements, user stories, technology stack justification, team roles, project timeline, risk assessment, and project setup confirmation (Trello, Slack, GitHub).

Our objective is to build a secure, scalable, and user-friendly healthcare platform that supports patients, doctors, and administrators with features such as appointment scheduling, patient record management, prescription handling, and administrative reporting.

This document is the foundation for all future sprints and provides clarity on what will be built, why it is important, and how it will be executed following Agile methodology.

2 Project Objectives

- **Design & Implementation:** Create a full-stack healthcare web application supporting multiple roles (patient, doctor, admin).
- **Agile Development:** Adopt Scrum practices — sprint planning, backlog grooming, daily standups, sprint retrospectives — to deliver incremental value.
- **Modern Technologies:** Utilize React.js for frontend, Node.js/Express for backend, MongoDB for database, and GitHub Actions for CI/CD.
- **Deployment Readiness:** Ensure the application can be deployed on a cloud platform (Heroku/AWS) with Docker-based containerization.
- **Documentation:** Maintain comprehensive documentation — requirements, design diagrams, API documentation, and final report.

3 Scope

In-Scope (Deliverable 1)

- Creation of this Requirements Document (10–15 pages)
- Git repository initialization with README
- Trello board setup with initial backlog cards
- Slack workspace creation for collaboration
- Technology stack selection and justification

Overall Project Scope

- Authentication & authorization with role-based access control
- Patient management: profile, history, medical documents

- Appointment scheduling: booking, cancellation, doctor confirmation
- Doctor dashboard: patient records, prescriptions, consultation notes
- Administrative features: user management, reports, system metrics
- Additional features: medical image upload, lab results, medication reminders

Out-of-Scope

- Integration with external hospital EHR systems
- HIPAA certification (best practices followed, no formal certification)
- Custom-built telemedicine/video engine (use third-party SDK if needed)

4 Actors and Roles

- **Patient:** Manage personal profile, book/cancel appointments, view medical history, access prescriptions, receive reminders.
- **Doctor:** Manage availability, view upcoming appointments, access patient records (with consent), add prescriptions, record consultation notes.
- **Admin:** Manage user accounts (patients/doctors), view reports, configure clinics, manage system-level data.
- **System (Automated):** Send notifications, process scheduled jobs (reminders), perform backups.

5 Functional Requirements

The functional requirements define **what the system must do** to meet user and business needs. Each requirement is grouped by module and described using clear, actionable language.

1. Authentication & Authorization

- The system shall allow users (patients, doctors, admins) to register with unique email addresses.
- The system shall verify user accounts via email confirmation before granting access.
- The system shall provide secure login/logout functionality using JWT-based authentication.
- The system shall allow users to reset their password using a secure email link.
- The system shall enforce role-based access control (RBAC) so each user can access only the features permitted for their role.

2. Patient Management

- The system shall allow patients to create and update their personal profile (name, date of birth, gender, contact info, address).
- The system shall allow patients to view their full medical history, including past appointments, prescriptions, and lab results.
- The system shall allow uploading of medical documents (PDF, images) and viewing them in a secure manner.
- The system shall provide patients with the ability to delete (or request deletion of) their data as per privacy guidelines.

3. Appointment Scheduling

- The system shall display doctor availability slots in a calendar view.
- Patients shall be able to book an appointment by selecting a doctor, date, and time.
- Patients shall be able to cancel or reschedule an appointment before the scheduled time.
- Doctors shall be notified of new appointment requests and shall have the ability to confirm or reject them.
- The system shall send email and in-app notifications to both patient and doctor upon booking, confirmation, or cancellation.

4. Doctor Dashboard

- Doctors shall be able to view a list of upcoming appointments and patient details.
- Doctors shall be able to update their availability (add or remove time slots).
- Doctors shall be able to create, edit, and share prescriptions with patients.
- Doctors shall be able to add consultation notes to a patient's medical history.

5. Administrative Features

- Admins shall be able to create, edit, or deactivate user accounts.
- Admins shall be able to assign or revoke roles (patient, doctor, admin).
- Admins shall be able to view system analytics: total number of patients, doctors, appointments by status, and active users.
- Admins shall be able to manage clinic/department data (names, locations, operating hours).

6. Notifications & Reminders

- The system shall send appointment reminders to patients via email one day before scheduled time.
- The system shall notify doctors of cancellations or reschedules immediately.
- The system shall provide a notifications panel where users can view past alerts.

7. Additional Features

- **Medication Reminder Dashboard:** Patients can log medications and receive reminders at specified times.
- **Basic Health Tips Section:** The home page shall display rotating health tips or awareness messages curated by admin.

6 Non-Functional Requirements

Non-functional requirements define **how the system should behave** to ensure usability, security, and maintainability.

1. Security

- All communication shall use HTTPS to ensure data confidentiality.
- Passwords shall be hashed using bcrypt or similar algorithm before storage.
- JWT tokens shall be used for authentication; refresh tokens will be rotated securely.
- User input shall be validated and sanitized to prevent SQL/NoSQL injection and XSS attacks.

2. Performance

- API response times shall average below 500ms under expected load.
- The system shall be able to handle at least 50 concurrent users during testing.
- Static assets shall be cached and minified to reduce page load times (<3s on standard broadband).

3. Scalability

- The backend shall be stateless so multiple instances can run behind a load balancer.
- The database shall use indexes on frequently queried fields (e.g., email, appointment date).

4. Availability

- The system shall aim for >95% uptime during development and demos.
- Automated daily backups of the database shall be taken.

5. Maintainability

- Code shall be organized in modules and follow consistent naming conventions.
- The project shall include a README, API documentation (Swagger/Postman), and environment setup guide.
- Critical functions shall have unit tests and continuous integration pipeline shall run tests on every push.

6. Privacy

- Patient data shall be accessed only by authorized roles.
- Sensitive information (e.g., prescriptions, history) shall never be logged in plain text.

7 User Stories with Acceptance Criteria

User stories capture the features from the perspective of end-users. Each story is prioritized (High/Medium/Low), given acceptance criteria, and assigned story points (1–8 scale). These stories form the initial product backlog for Sprint 1 and beyond.

User Story	Acceptance Criteria	Priority	SP
As a patient, I can sign up so I can access the system	• Account is created and saved to database. • Verification email is sent with activation link. • System prevents duplicate emails.	High	3
As a user, I can log in so I can access protected features	• Login returns JWT token for authenticated requests. • Invalid credentials return error message. • Session expires after defined timeout.	High	2

As a user, I can reset my password	• Password reset email sent within 1 minute. • New password must meet complexity rules. • Old password no longer valid.	High	3
As a patient, I can manage my profile	• Create, view, edit, delete (soft-delete) profile fields. • Changes reflected immediately on dashboard. • Unauthorized users cannot access others' profiles.	High	3
As a doctor, I can define my availability	• Doctor can add/remove time slots. • Patients can only see available slots. • Conflicting slots are prevented.	High	3
As a patient, I can book an appointment	• Appointment is saved in DB and visible in patient dashboard. • Doctor receives notification. • Double-booking for same slot is not allowed.	High	5
As a doctor, I can confirm or cancel appointments	• Doctor updates status (confirmed/cancelled). • Patient receives notification immediately. • Updated status reflected in both dashboards.	High	3

As a doctor, I can create prescriptions	• Prescription linked to appointment. • Patient can download or view prescription. • Data saved securely to database.	Medium	3
As an admin, I can manage users	• Admin can add/remove users. • Admin can update user roles. • Changes take effect immediately.	High	4
As a user, I can upload medical documents	• Secure upload (PDF, JPG, PNG). • Preview available in browser. • Only owner/doctor can view file.	Medium	5
As an admin, I can view analytics	• Dashboard shows user counts, appointment stats. • Data refreshes in near real-time. • Admin-only access enforced.	Medium	4
As a patient, I can receive medication reminders	• Reminder sent at correct scheduled time. • Reminders appear in notification panel and via email.	Low	3

Note: High-priority stories will be included in Sprint 1 and Sprint 2. Medium and Low priority stories may be scheduled in later sprints depending on velocity.

8 Technology Stack and Justification

This project adopts a modern, scalable, and developer-friendly technology stack selected to balance learning outcomes and project complexity.

Frontend

Chosen Technology: React.js (with Vite or CRA)

- **Pros:**
  - Component-based architecture for reusable UI pieces.
  - Large ecosystem of libraries (React Router, Redux, Material UI).
  - Strong community support and documentation.
- **Cons:**
  - Requires setup for state management (but Context API/Redux available).
  - Requires bundling (Vite solves performance overhead).
- **Reason for Choice:** React offers a gentle learning curve, wide adoption, and flexibility for building a responsive and interactive UI quickly.

Backend

Chosen Technology: Node.js with Express.js

- **Pros:**
  - Same language (JavaScript) across frontend and backend, reducing cognitive load.
  - Non-blocking event-driven architecture — excellent for I/O heavy apps like appointment scheduling.
  - Mature ecosystem with npm packages for auth, file upload, testing.
- **Cons:**
  - Requires careful error handling (async/await).
  - Not strongly typed (can mitigate with TypeScript if needed).
- **Reason for Choice:** Speed of development, flexibility, and easy deployment on cloud services.

Database

Chosen Technology: MongoDB (NoSQL) with Mongoose ORM

- **Pros:**
  - JSON-like document storage fits dynamic healthcare data.
  - Easy to evolve schema without complex migrations.
  - Powerful query capabilities and good integration with Node.js.
- **Cons:**
  - Less suited for complex transactions (can be mitigated with careful design).
- **Reason for Choice:** Flexibility is critical for evolving requirements (medical notes, images, appointments). Mongoose provides validation and relationships for data integrity.

Deployment and DevOps

- **Docker:** Standardize development and deployment environments.
- **Heroku/AWS Free Tier:** Simple first deployment target with student credits.
- **GitHub Actions:** Automate build/test pipeline for each commit.

Collaboration Tools

- **GitHub:** Version control with branching strategy (feature branches + pull requests).
- **Trello:** Kanban board for backlog management and sprint tracking.
- **Slack:** Team communication, standups, and tool integrations (GitHub/Trello notifications).

9 Team Roles and Responsibilities

Our team consists of three members. To ensure balanced workload and smooth collaboration, each member is assigned multiple roles as shown below:

Role	Assigned Member	Key Responsibilities
Scrum Master / Project Lead	Sajid Shahriar Sanam	Facilitates sprint planning, daily standups, and retrospectives. Maintains Trello board, coordinates communication with instructor, ensures timely submission of deliverables.

Frontend Developer	Israt Zannat	Develops React components, handles routing and state management, integrates APIs, ensures responsive design and accessibility.
Backend Developer	Shajeda Isnat	Implements REST APIs with Node.js/Express, manages authentication and database models, writes unit tests, ensures security and scalability.
DevOps / Deployment	Israt Zannat	Sets up Docker, manages local development environment, prepares cloud deployment pipelines, maintains environment configuration files.
QA / Tester	Sajid Shahriar Sanam	Creates and runs test cases, validates acceptance criteria for user stories, documents bugs and works with devs to resolve them.
Documentation Owner	Shajeda Isnat	Maintains project documentation, prepares deliverables (requirements, design, final report), keeps README and Trello descriptions updated.

Note: Since the team is small, members are cross-functional and may support each other in overlapping areas during sprints to ensure smooth progress.

10 Project Timeline and Milestones

The project will be completed over approximately 12 weeks, following the deliverable schedule provided by the instructor. Each deliverable builds on the previous one, following Agile incremental delivery.

Week	**Deliverable**	**Due Date**	**Key Components / Description**
2	D1: Requirements	Sept 17, 2025	Requirements document (10–15 pages), technology stack selection with justification, project setup (Git, Trello, Slack). Serves as the foundation for future sprints.
4	D2: Design	Oct 5, 2025	System architecture diagrams, UI/UX wireframes, database schema design. Sets the blueprint for development.

6	D3: Backend MVP	Oct 19, 2025	Implementation of REST API endpoints, user authentication, database integration. Focus on backend functionality and testing.
8	D4: Frontend Integration	Nov 2, 2025	React frontend implementation, connecting to backend APIs, building forms and dashboards, basic styling.
10–12	D5: Deployment & Final	Nov 16 & 30, 2025	Containerization (Docker), deployment to cloud (Heroku/AWS), final testing, optimization, documentation, and recorded video demo submission.

Note: Each deliverable will be accompanied by a short sprint retrospective and updated Trello board snapshot to show progress.

11 Risk Assessment and Mitigation

Effective risk management is critical to ensure timely delivery. Risks are categorized by probability (Low/Medium/High) and impact (Low/Medium/High).

Risk	**Probability**	**Impact**	**Mitigation Strategy**
Team member unavailability	Med	High	Divide tasks into smaller stories, maintain a shared Trello board so others can take over if needed. Buffer days built into timeline.
Unclear or changing requirements	High	Med	Maintain updated product backlog, confirm requirements with instructor before sprint start, document all changes.
API contract mismatch (frontend/backend)	Med	Med	Define API schema early (Postman/Swagger), use mock APIs during frontend dev, integrate continuously.
File upload/storage complexity	Med	Med	Start with local storage, later switch to S3/MinIO, restrict file types/-sizes, test with sample data early.
Security vulnerabilities	Med	High	Use bcrypt hashing, JWT tokens, input validation, regular code reviews, security checklist before deployment.

Deployment/configuration issues	Low	High	Prepare Docker + docker-compose early, deploy staging version in Week 6 to avoid last-minute surprises.

Overall Approach: Risks will be tracked on Trello as separate cards, reviewed in weekly standups, and mitigated proactively during sprint planning.

12 Setup Confirmation

The following initial setup tasks have been completed as part of Deliverable 1:

- **Trello Board:** Created *"Healthcare Web Application — Sprint Board"* with lists: Backlog, To Do, In Progress, Review, Done. Initial backlog cards added.

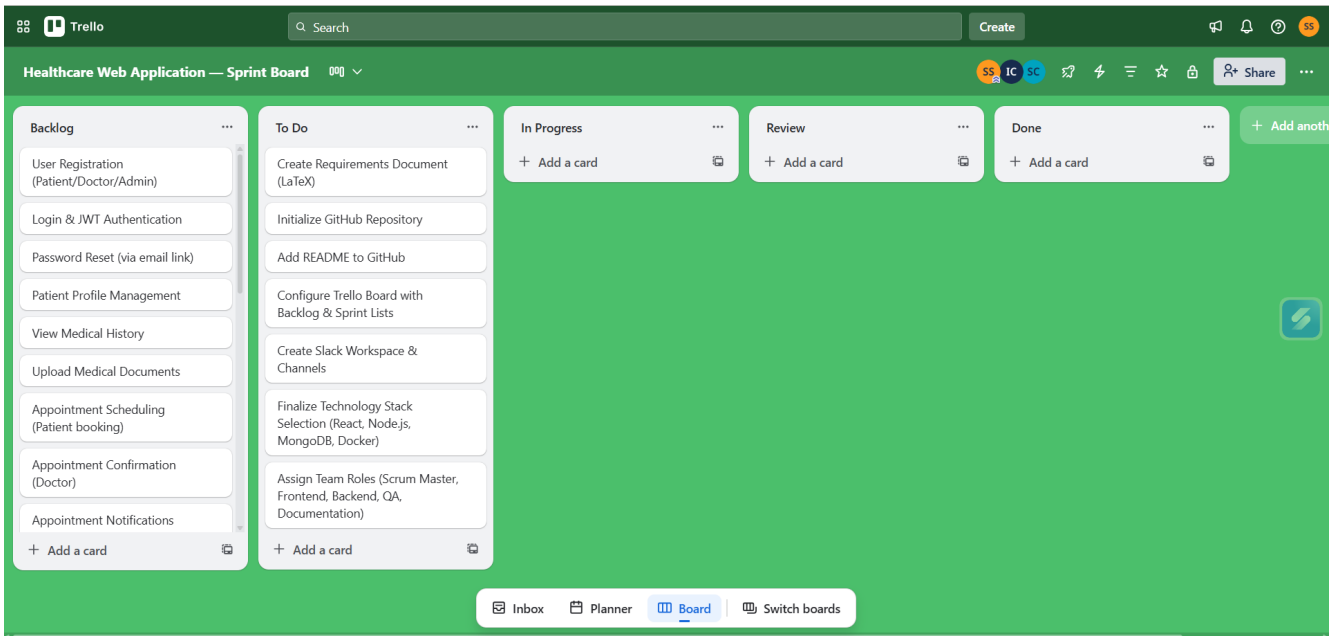

Figure 1: Trello Board Setup Screenshot

- **Slack Workspace:** Created dedicated workspace with channels #general, #frontend, #backend, #devops, #standups.

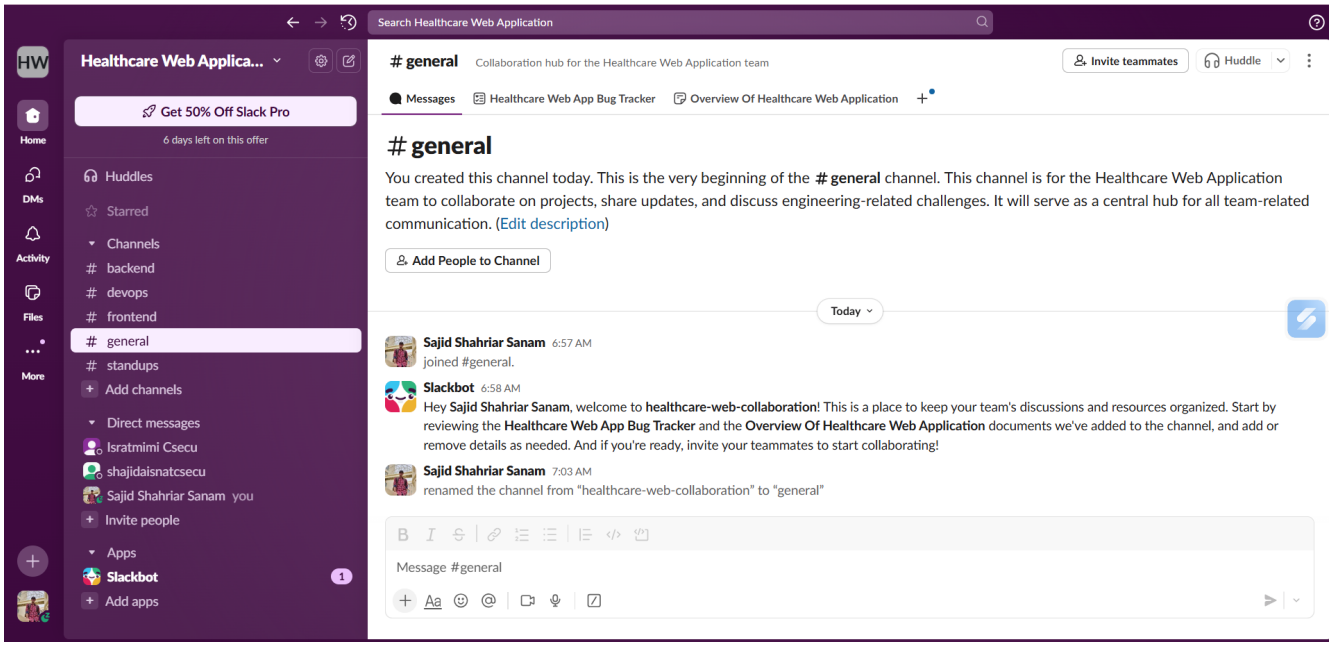

Figure 2: Slack Setup Screenshot

- **GitHub Repository:** Repository initialized with README and project folders (frontend, backend). Branch protection rules enabled for main branch.

  The repository is available at: https://github.com/SajidShahriarSanam/healthcare-web-application.git

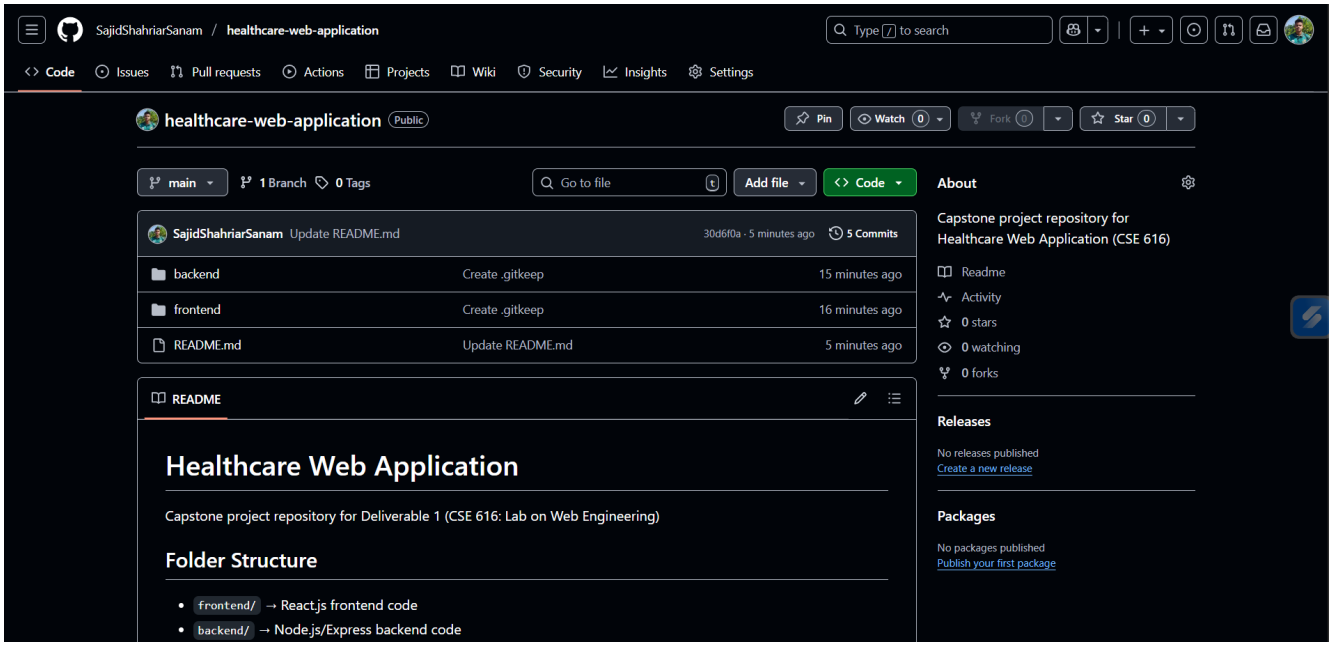

Figure 3: GitHub Repository Setup Screenshot

These tools will be used for ongoing sprint tracking, communication, and collaboration throughout the project.